



Tìm hiểu Apache Spark và ứng dụng thuật toán Random Forest

Phùng Ngọc Bảo - 2212318

Nguyễn Công Đạt - 2214025

Nội dung

01

Mục tiêu

02

Apache Spark

03

Kết nối cụm

04

Word count

05

Random Forest

06

Cải thiện hiệu suất

07

Kết luận

01

Mục tiêu

Các mục tiêu chính

Trình bày tổng quan về nền tảng Apache Spark và kiến trúc xử lý song song phân tán.

Phân tích nguyên lý hoạt động của thuật toán Random Forest trong bài toán học máy tuần tự cũng như song song.

Ứng dụng Random Forest trên Apache Spark để mô phỏng quá trình xử lý dữ liệu lớn và đánh giá hiệu suất, đặc biệt là về thời gian xử lý với dữ liệu lớn.

Dự án này không chỉ nhằm mục đích chạy một thuật toán, mà còn nhằm mục đích nắm vững và minh họa cơ chế kiến trúc phân tán cơ bản đứng sau các công nghệ Big Data hiện đại.

What is Apache Spark?

Its Definition, Features, and Uses for Data Analytics



02

Apache Spark

Khái niệm và vai trò

Spark là gì? Một khung xử lý phân tán mã nguồn mở, thống nhất, được thiết kế để xử lý dữ liệu lớn (Big Data) một cách nhanh chóng và hiệu quả.

Điểm mạnh: Xử lý trong bộ nhớ (In-Memory Processing), giúp tốc độ nhanh hơn Hadoop MapReduce truyền thống (có thể nhanh hơn 10x-100x).

Mục tiêu: Cung cấp một nền tảng thống nhất cho nhiều loại tác vụ dữ liệu lớn, bao gồm: ETL, học máy, xử lý đồ thị và xử lý luồng (streaming).

Kiến trúc nổi bật

RDD (Resilient Distributed Dataset): Cấu trúc dữ liệu cơ bản của Spark, cho phép lưu trữ tập dữ liệu lớn và phân tán trên các node, có khả năng chịu lỗi.

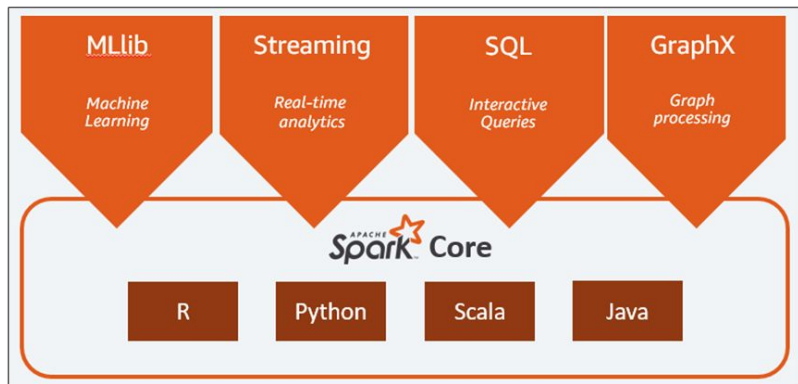
Spark Core: Cơ chế lập lịch, tối ưu hóa và thực thi các tác vụ phân tán.

Chế độ hoạt động: Spark có thể chạy trên nhiều Cluster Manager khác nhau (như YARN, Mesos) hoặc chế độ độc lập (Standalone).

Các module chính

Spark cung cấp một bộ thư viện phong phú, cho phép người dùng thực hiện nhiều loại tác vụ chỉ trên một framework duy nhất:

- **Spark SQL:** Xử lý dữ liệu có cấu trúc bằng SQL và DataFrames/Datasets.
- **Spark Streaming/Structured Streaming:** Xử lý dữ liệu theo thời gian thực (real-time) hoặc luồng dữ liệu (streaming data).
- **MLlib:** Thư viện học máy có khả năng mở rộng (scalability).
- **GraphX:** Thư viện xử lý đồ thị (graph processing).



Kiến trúc

Driver Program <-> Cluster Manager:

- **Mục đích:** Driver (chứa **SparkContext**) kết nối với Cluster Manager (ví dụ: YARN, Mesos) để **yêu cầu tài nguyên** (cụm).

Cluster Manager <-> Worker Nodes:

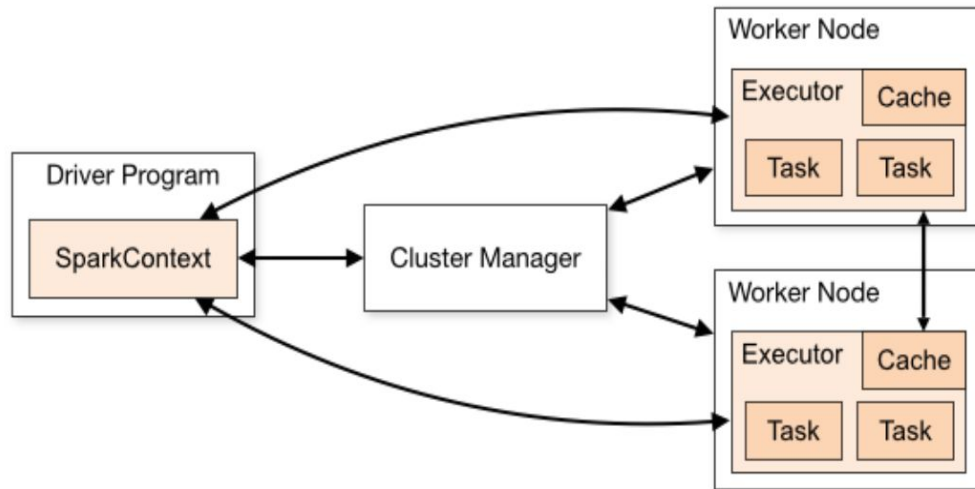
- **Mục đích:** Cluster Manager nhận yêu cầu và **khởi chạy các Executor** trên các Worker Node.

Driver Program <-> Executor:

- **Mục đích:** Driver trực tiếp giao tiếp với các Executor để **gửi các Task (Nhiệm vụ)** và **giám sát** quá trình thực thi.

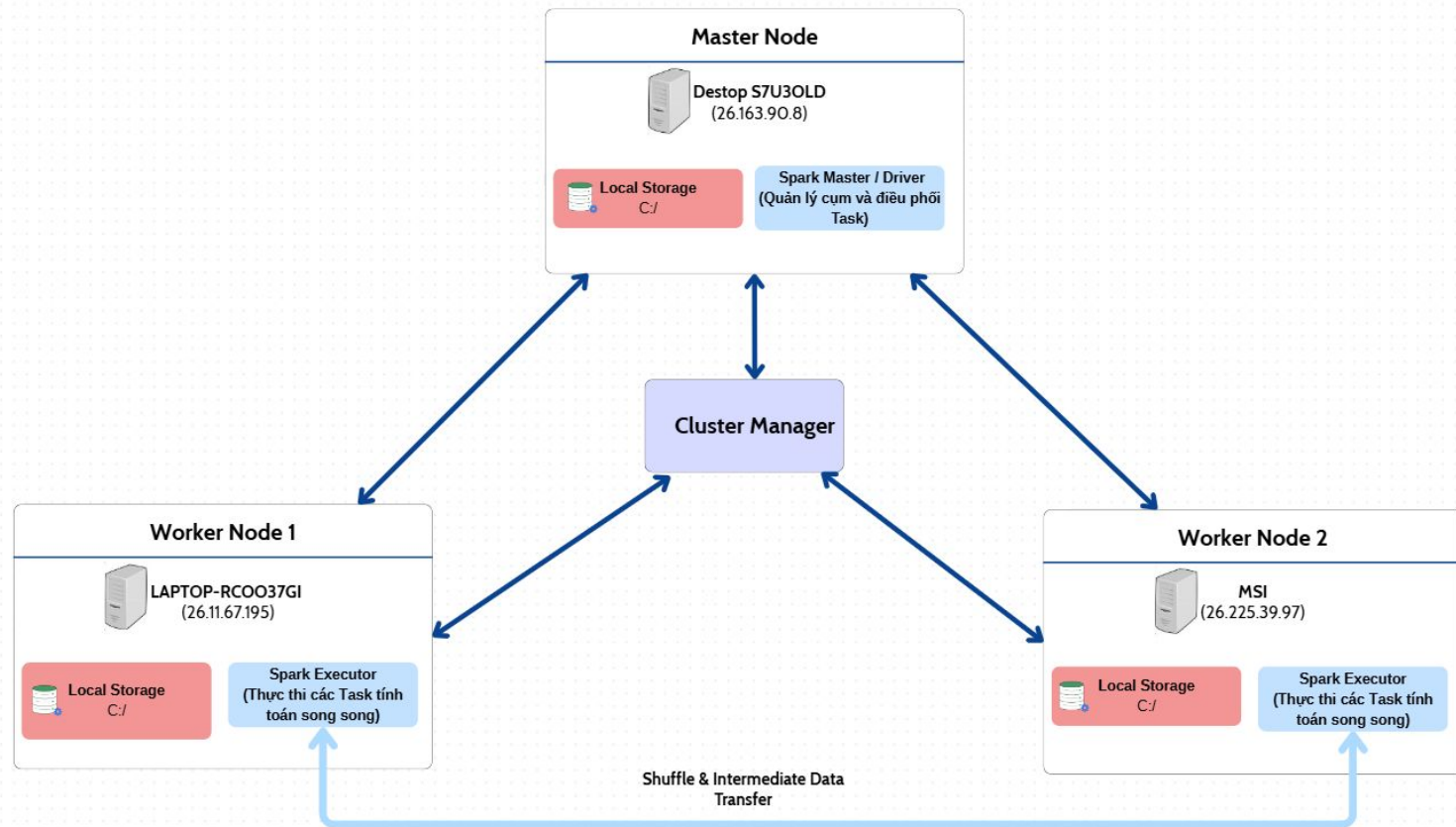
Executor <-> Executor:

- **Mục đích:** Khi xảy ra **Shuffle** (giai đoạn trộn/phân phối dữ liệu lại), các Executor trao đổi dữ liệu trung gian trực tiếp với nhau.



3. Kết nối cụm

Thiết kế kết nối cụm



Thiết kế kết nối cụm

Node	IP	Cores	Memory
Master	26.163.90.8	-	-
Worker 1	26.11.67.195	8	6.9
Worker 2	26.225.39.97	16	30.7



4. Word Count



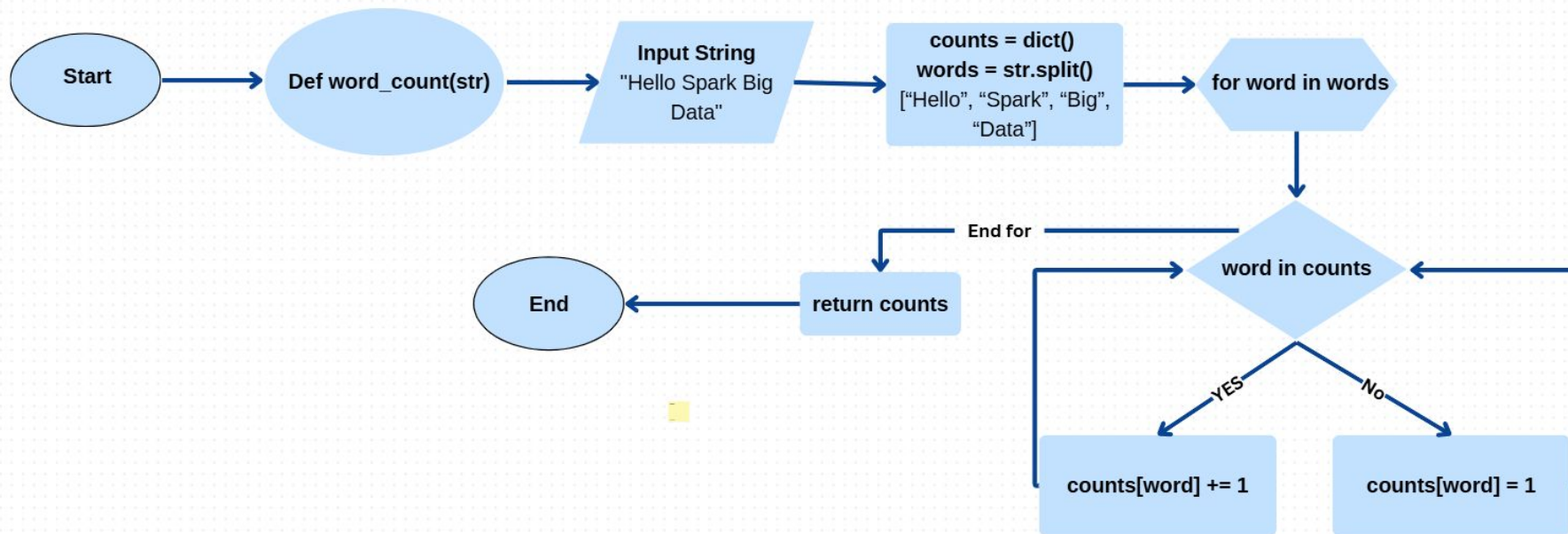
Giới thiệu Word Count (Đếm chữ)

Định nghĩa: Word Count là thuật toán cơ bản, dùng để **đếm tần suất (số lần xuất hiện)** của mỗi từ duy nhất trong một tập dữ liệu văn bản lớn.

Mục đích: Là bài toán kinh điển và đơn giản nhất để giới thiệu các nguyên tắc hoạt động của các khung xử lý dữ liệu lớn (Big Data Frameworks).

Kết quả: Một danh sách các cặp (Từ, Số lần đếm) cuối cùng.

Đếm chữ với mô hình tuần tự



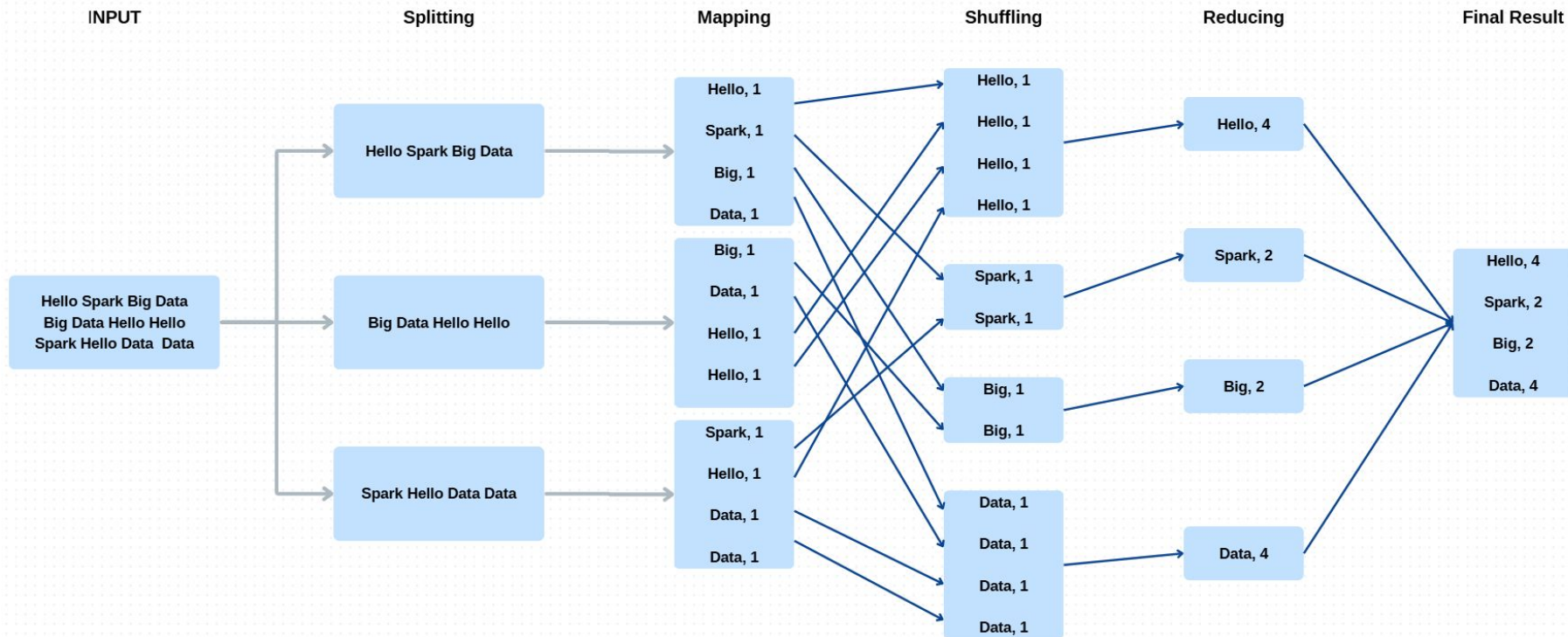
Mô hình xử lý phân tán

Word Count được thực hiện hiệu quả nhất trên kiến trúc phân tán như **MapReduce** (Hadoop) hoặc **Apache Spark**.

- **Chia nhỏ (Splitting/Input):** Tập tin văn bản lớn được chia thành các khối nhỏ (Partition/Split).
- **Xử lý độc lập:** Các khối dữ liệu này được gửi đến các máy con (Worker Nodes) để xử lý **song song**.

Đếm chữ với mô hình song song

The overall MapReduce Work Count Process



So sánh Word Count khi sử dụng Spark với không sử dụng:

Thời gian chạy tuần tự	Thời gian chạy song song (2 workers node – 24 cores)	Thời gian chạy song song (1 worker node – 8 cores)
0.0000	8.0992	8.2484

```
=====
PHẦN 1: PYTHON LOCAL (CHỈ CHẠY TRÊN MÁY MASTER)
=====
✅ SỐ TỪ KHÁC NHAU (UNIQUE): 381
⬤ THỜI GIAN PYTHON: 0.0000 giây

--- Top 10 từ (Python): ---
'the': 38
'a': 28
'of': 25
'word': 24
'and': 23
'words': 21
'is': 19
'to': 18
'count': 11
'in': 11
```

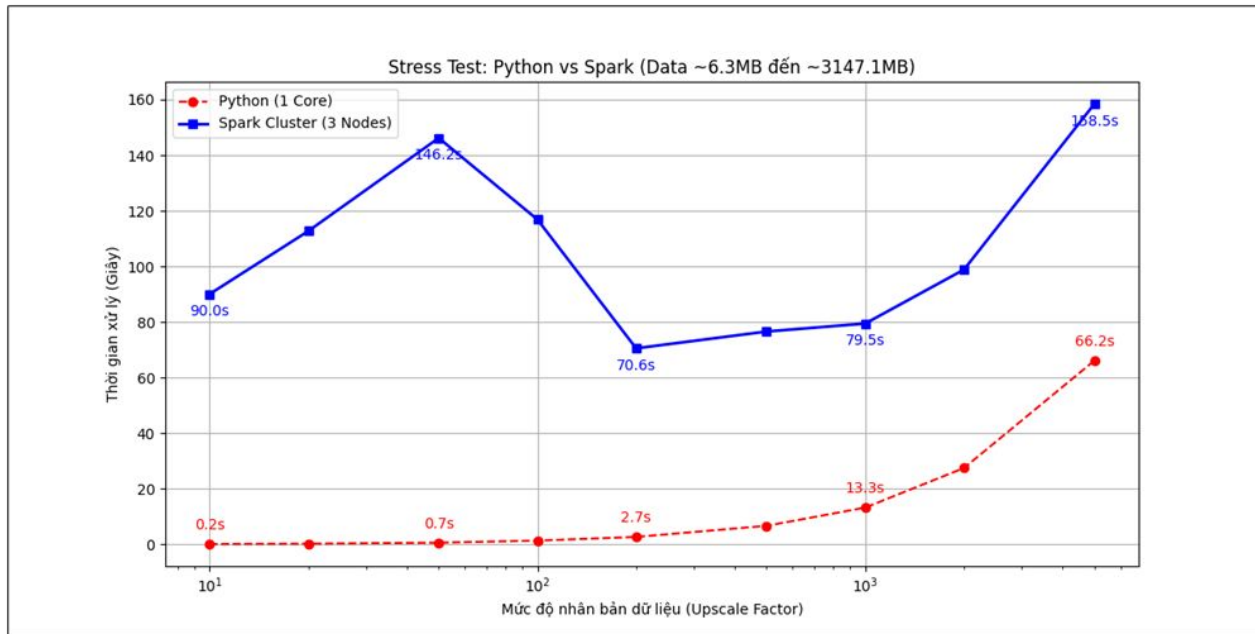
```
=====
PHẦN 2: SPARK CLUSTER (PHẦN TÁN TRÊN 2 MÁY WORKER)
=====
Dang khoi tao Spark...
Picked up _JAVA_OPTIONS: --add-opens=java.base/java.
Picked up _JAVA_OPTIONS: --add-opens=java.base/java.
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel)
25/12/11 10:05:43 WARN SparkConf: Note that spark.lo
✅ SỐ TỪ KHÁC NHAU (UNIQUE): 381
⬤ THỜI GIAN SPARK: 8.0992 giây

--- Top 10 từ (Spark Cluster): ---
'the': 38
'a': 28
'of': 25
'word': 24
'and': 23
'words': 21
'is': 19
'to': 18
'count': 11
'in': 11
```

```
=====
PHẦN 2: SPARK CLUSTER (PHẦN TÁN TRÊN 1 MÁY WORKER)
=====
Dang khoi tao Spark...
Picked up _JAVA_OPTIONS: --add-opens=java.base/java
Picked up _JAVA_OPTIONS: --add-opens=java.base/java
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel)
25/12/11 10:09:37 WARN SparkConf: Note that spark.l
✅ SỐ TỪ KHÁC NHAU (UNIQUE): 381
⬤ THỜI GIAN SPARK: 8.2484 giây

--- Top 10 từ (Spark Cluster): ---
'the': 38
'a': 28
'of': 25
'word': 24
'and': 23
'words': 21
'is': 19
'to': 18
'count': 11
'in': 11
```

So sánh Word Count với độ lớn dữ liệu tăng dần



Nguyên nhân chính nằm ở chi phí khởi tạo và độ trễ giao tiếp mạng của Spark là quá lớn so với thời gian tính toán thực tế. Trong khi Python tận dụng tối đa tốc độ truy xuất trực tiếp của RAM trên đơn máy, Spark lại tiêu tốn tài nguyên cho việc quản lý Executors và quá trình Shuffle dữ liệu giữa các node. Điều này khẳng định rằng công nghệ phân tán chỉ thực sự trở nên cần thiết khi dữ liệu vượt quá giới hạn phần cứng của một máy đơn lẻ (Big Data), còn với dữ liệu nằm trong khả năng của RAM, giải pháp tuần tự vẫn là tối ưu nhất.



5. Random Forest



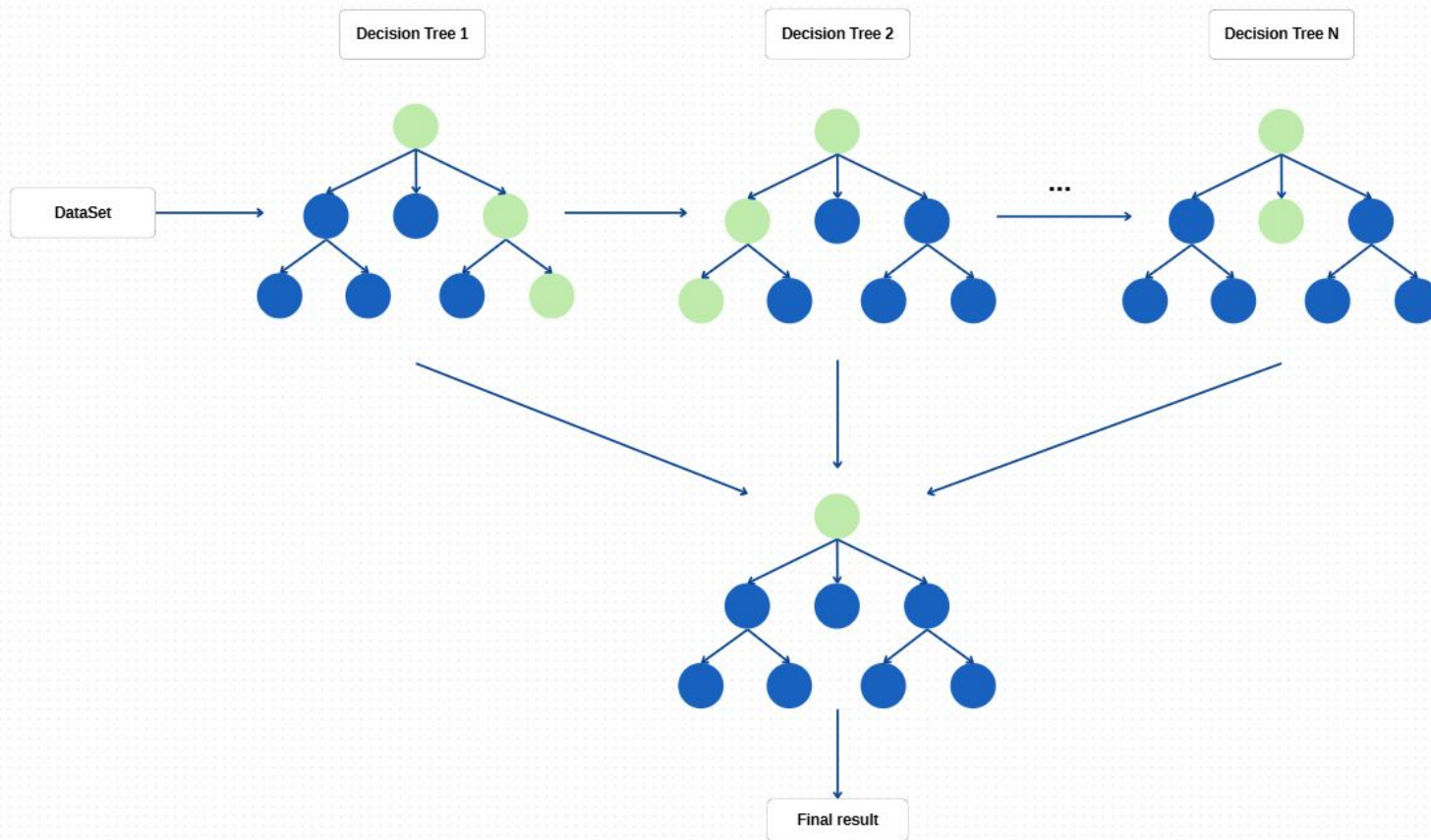
Giới thiệu Random Forest (Rừng ngẫu nhiên)

Định nghĩa: Random Forest là một thuật toán học máy kết hợp (Ensemble Learning) mạnh mẽ, được sử dụng cho cả bài toán **Phân loại (Classification)** và **Hồi quy (Regression)**.

Cơ chế: Nó hoạt động bằng cách xây dựng một lượng lớn (Forest) các **Cây Quyết định (Decision Trees)** riêng lẻ.

Mục tiêu: Giảm thiểu hiện tượng **Overfitting (Quá khớp)** và cải thiện độ chính xác (Accuracy) so với một cây quyết định đơn lẻ.

RF với mô hình tuần tự



Ưu điểm Random Forest (Rừng ngẫu nhiên)

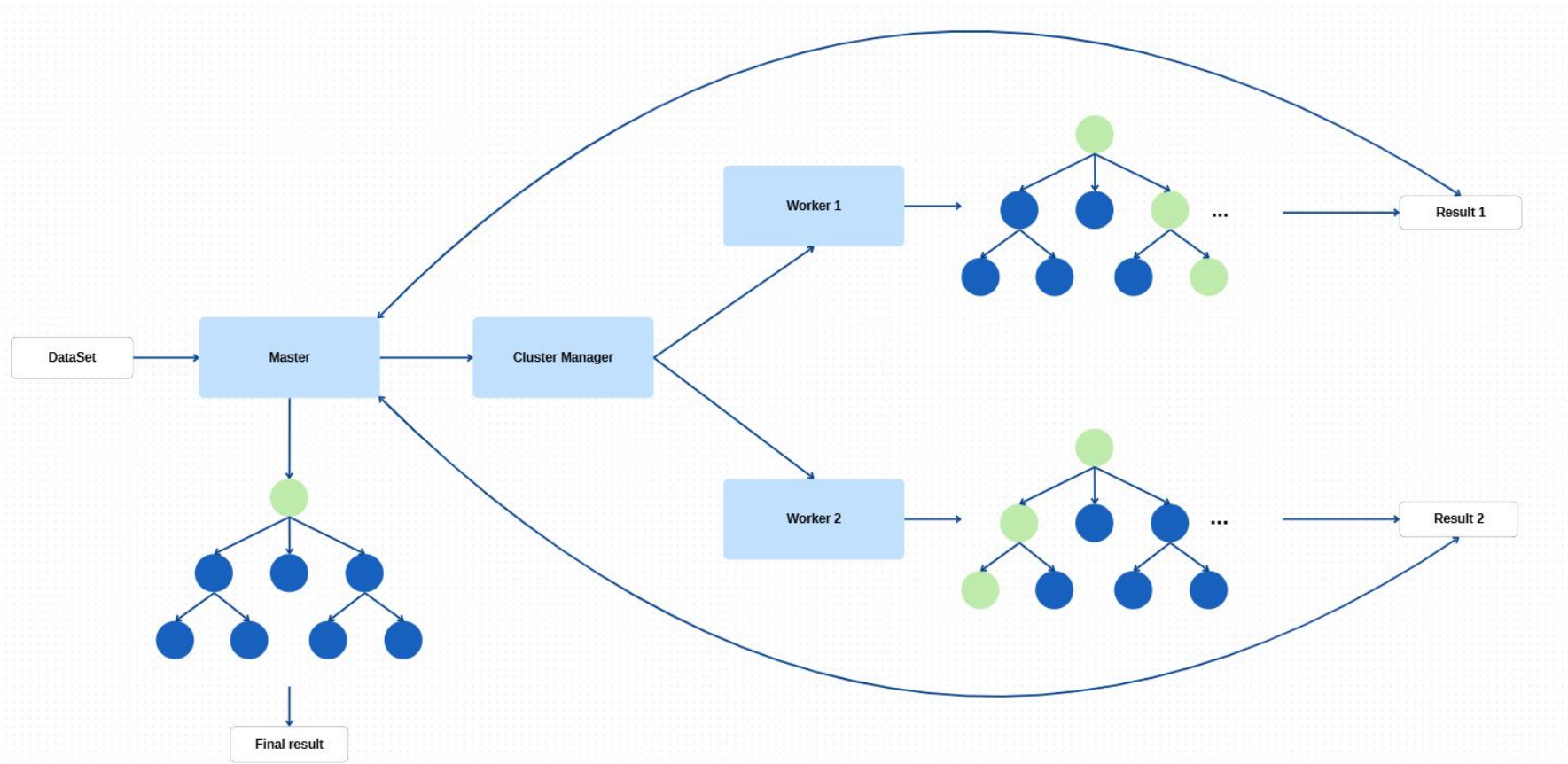
Độ chính xác cao: Thường cho kết quả tốt hơn nhiều thuật toán đơn lẻ khác.

Chống Overfitting: Nhờ tính ngẫu nhiên và kỹ thuật Bagging.

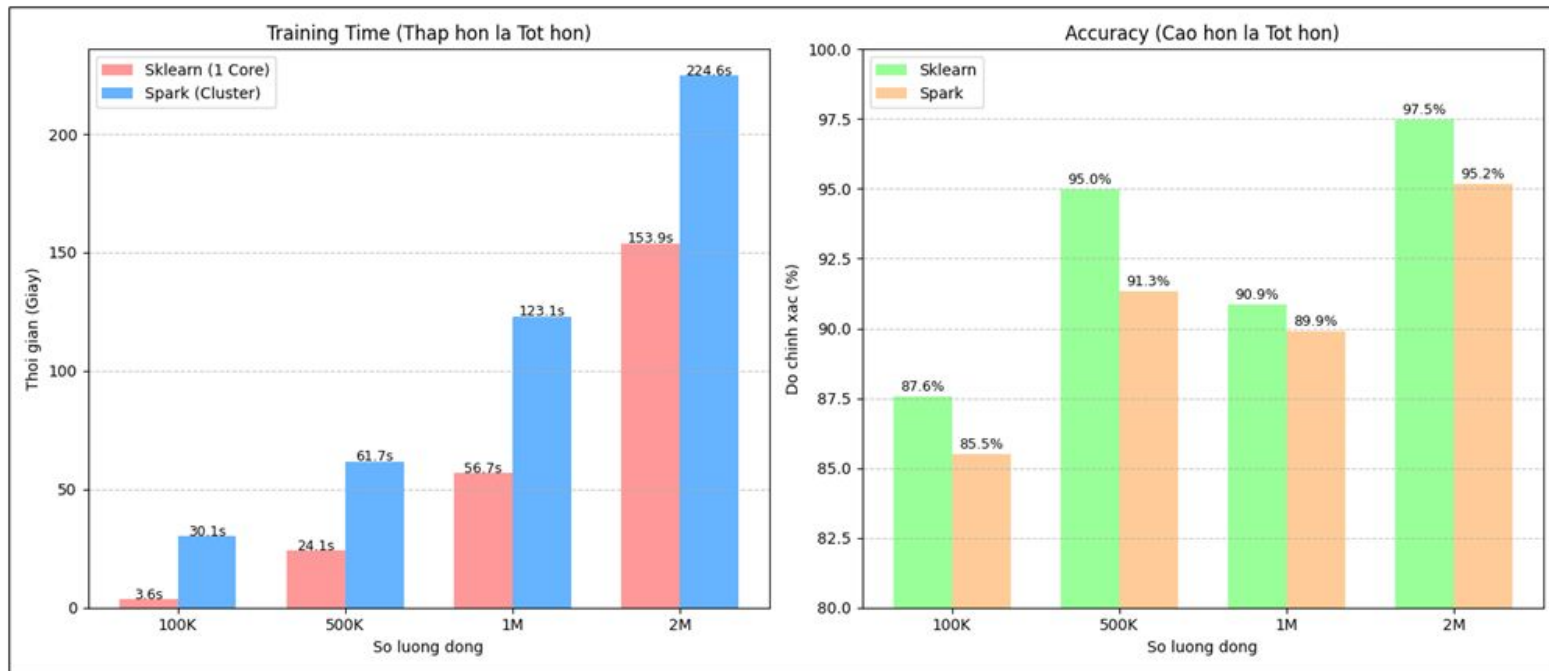
Xử lý Dữ liệu Lớn: Khả năng mở rộng tốt do tính chất hoạt động song song.

Ít cần tinh chỉnh: Hiếm khi cần chuẩn hóa (Normalization) dữ liệu và ít bị ảnh hưởng bởi dữ liệu ngoại lai (Outliers).

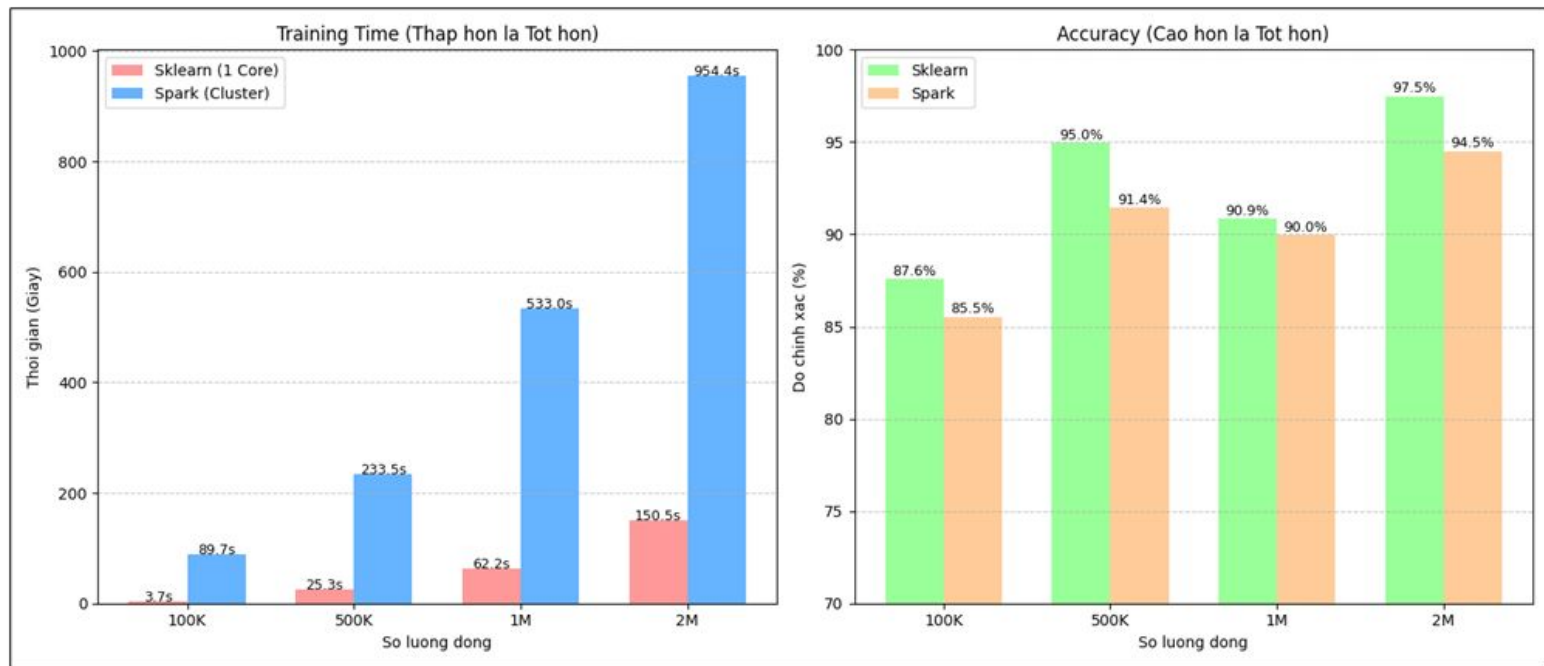
RF với mô hình song song



RF với mô hình song song (1 worker)



RF với mô hình song song (2 workers)



Nhận xét chung

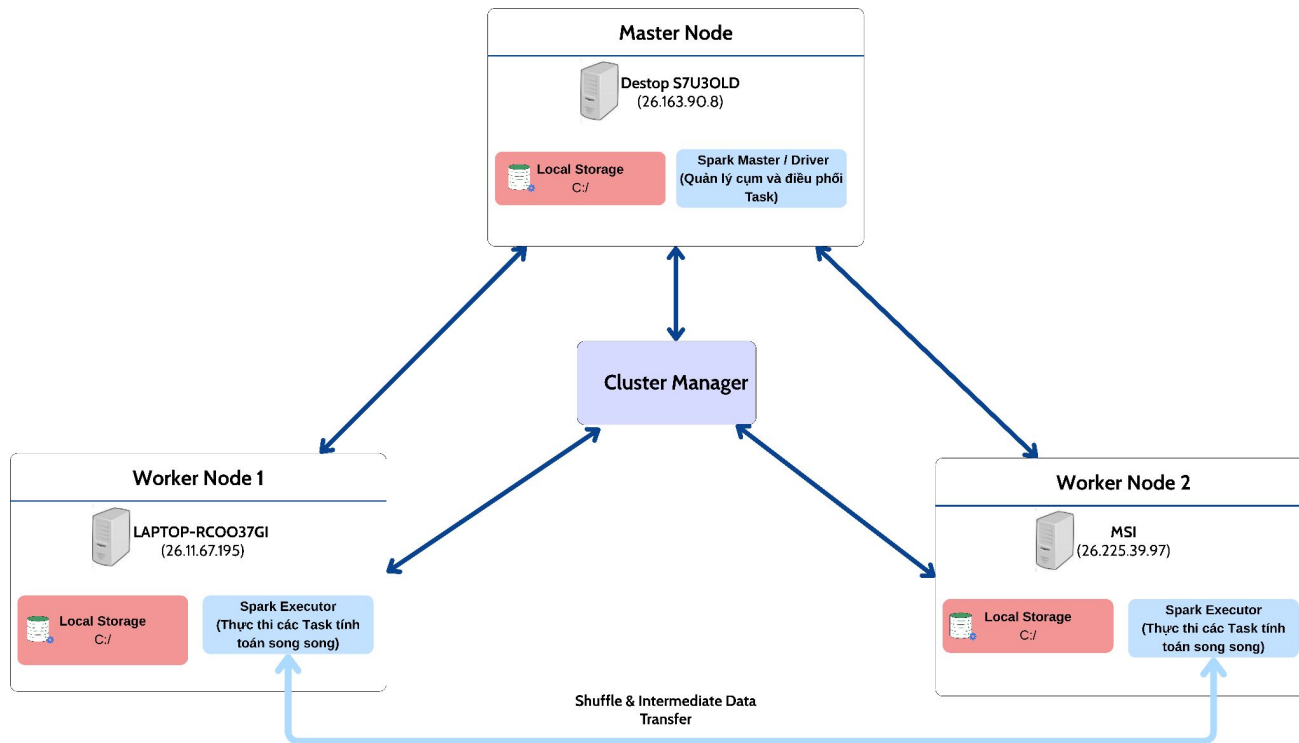
Độ lớn (dòng)	Thời gian chạy tuần tự lần 1	Thời gian chạy tuần tự lần 2	Thời gian chạy song song với Spark (cụm 1 worker node)	Thời gian chạy song song với Spark (cụm 2 workers node)
100000	3.6	3.71	30.1	89.74
500000	24.1	24.33	61.7	233.53
1000000	56.7	62.18	123.1	533.01
2000000	153.9	150.46	224.6	954.40

Khẳng định rằng với lượng dữ liệu chưa vượt quá giới hạn bộ nhớ của một máy đơn, chi phí giao tiếp mạng (network overhead) để điều phối và xáo trộn dữ liệu (shuffling) giữa các node trong mô hình phân tán đã trở thành nút thắt cổ chai, khiến hiệu năng tổng thể thấp hơn nhiều so với việc xử lý cục bộ.

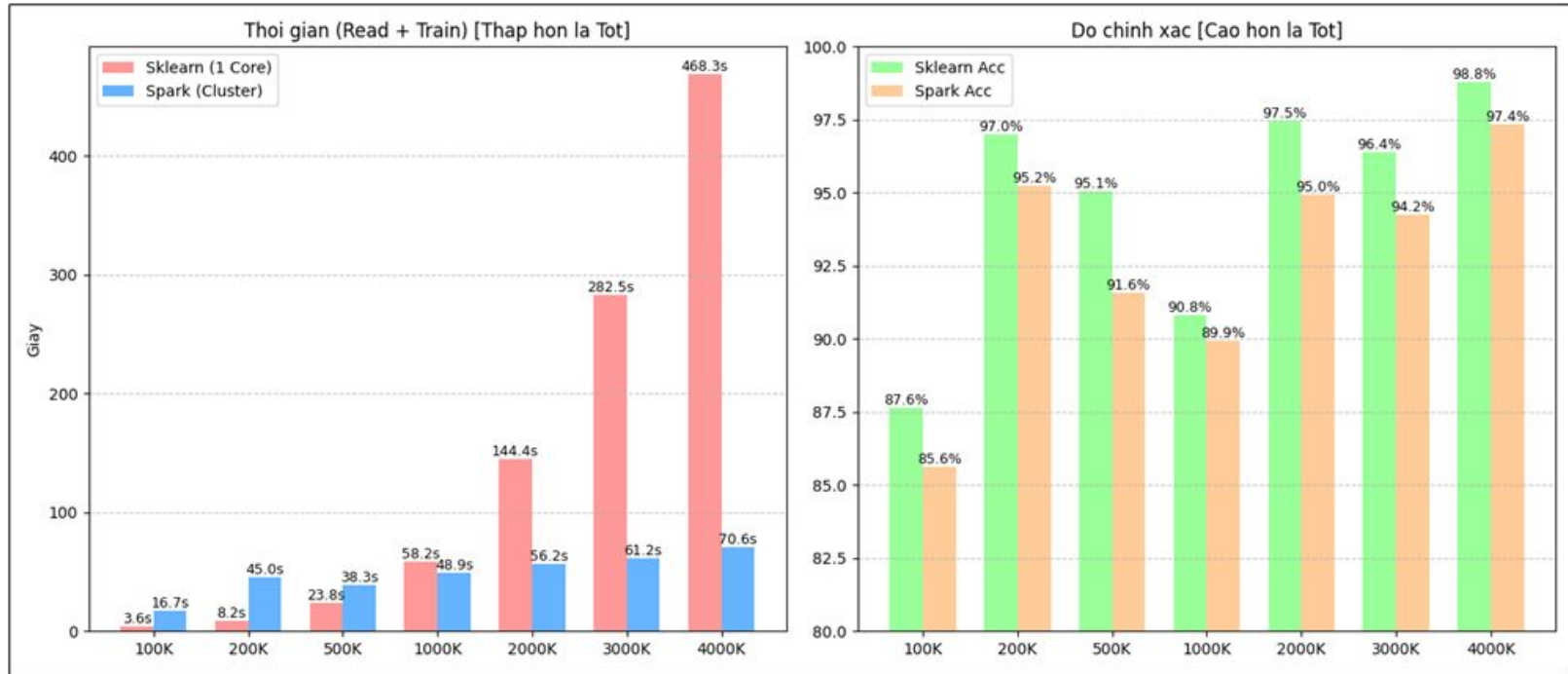
6. Cải thiện hiệu suất

Data Mirroring

Dữ liệu được đưa thủ công đến các nút.



Data Mirroring



Điều này phản ánh rõ chi phí “đầu vào” khi sử dụng Spark (khởi tạo phiên làm việc, phân tán dữ liệu, shuffle giữa các node) lớn hơn rất nhiều so với chi phí tính toán bản thân mô hình, nên ở quy mô nhỏ, xử lý trên một máy đơn là lựa chọn hiệu quả hơn.

Sự cần thiết của HDFS

Nội dung	Điểm Chính
Tiêu đề	HDFS: Nền tảng Lưu trữ cho Big Data
Vấn đề	Hệ thống file cục bộ (Local Filesystem) không thể xử lý dữ liệu kích thước lớn (TB/PB) và không có khả năng chịu lỗi.
Giải pháp	HDFS là một hệ thống file phân tán, được thiết kế để lưu trữ các tập tin cực lớn trải rộng trên hàng ngàn máy chủ.
Vai trò trong đề án	HDFS là nơi lưu trữ dữ liệu thô (raw data) cho các thuật toán của chúng ta (Word Count, Random Forest).

HDFS lưu trữ dữ liệu phân tán

Browse Directory

/DataTest

Go!

Show

25

entries

Search:

☐	⌵ Permission	⌵ Owner	⌵ Group	⌵ Size	⌵ Last Modified	⌵ Replication	⌵ Block Size	⌵ Name	⌵
☐	drwxr-xr-x	ADMIN	supergroup	0 B	Dec 11 15:21	0	0 B	data_100000.parquet	
☐	drwxr-xr-x	ADMIN	supergroup	0 B	Dec 11 15:23	0	0 B	data_1000000.parquet	
☐	drwxr-xr-x	ADMIN	supergroup	0 B	Dec 11 15:21	0	0 B	data_200000.parquet	
☐	drwxr-xr-x	ADMIN	supergroup	0 B	Dec 11 15:24	0	0 B	data_2000000.parquet	
☐	drwxr-xr-x	ADMIN	supergroup	0 B	Dec 11 15:26	0	0 B	data_3000000.parquet	
☐	drwxr-xr-x	ADMIN	supergroup	0 B	Dec 11 15:29	0	0 B	data_4000000.parquet	
☐	drwxr-xr-x	ADMIN	supergroup	0 B	Dec 11 15:22	0	0 B	data_500000.parquet	

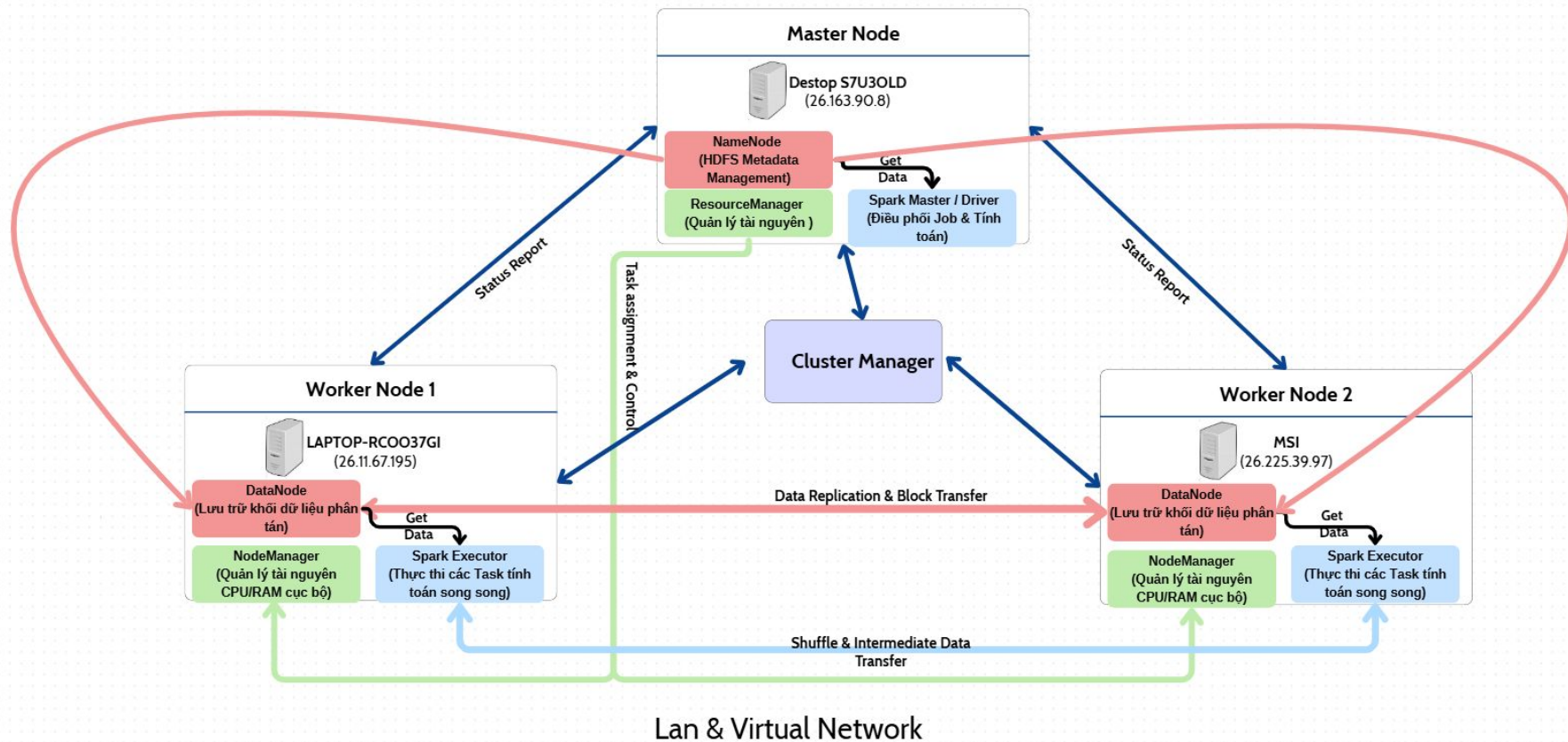
Showing 1 to 7 of 7 entries

Previous

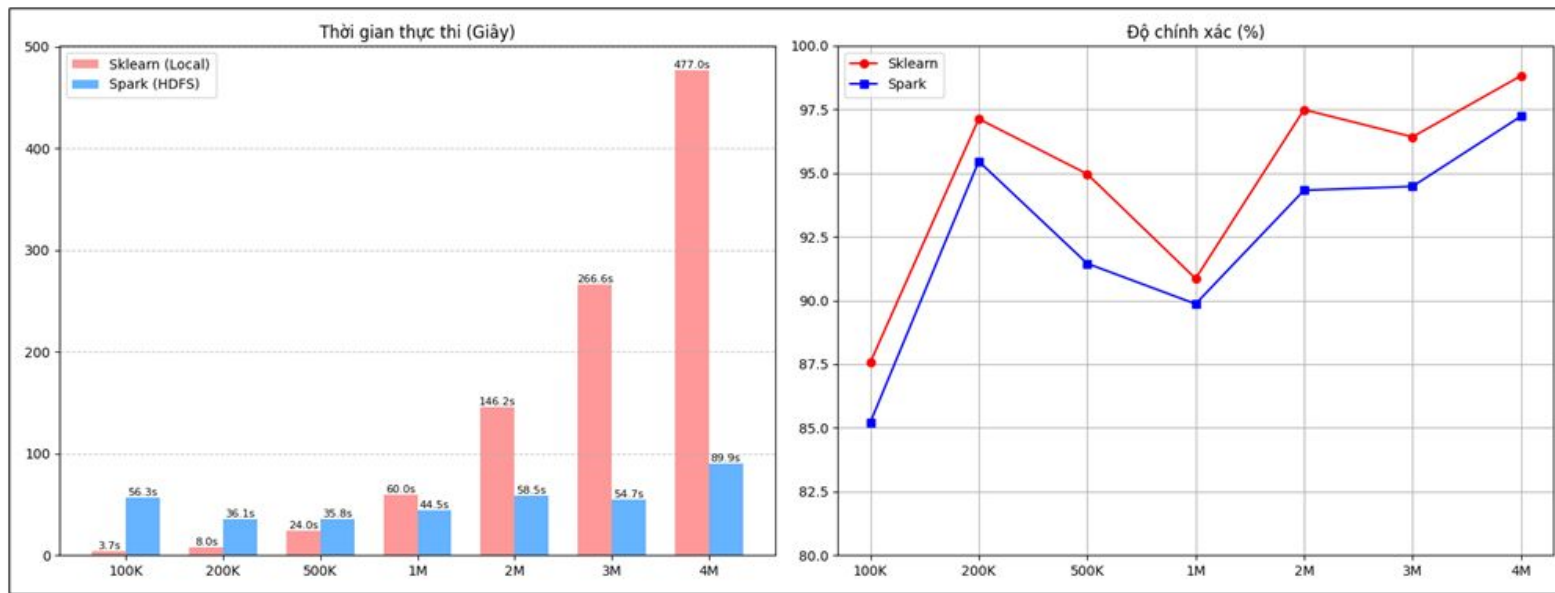
1

Next

Sơ đồ mô hình ứng dụng HDFS

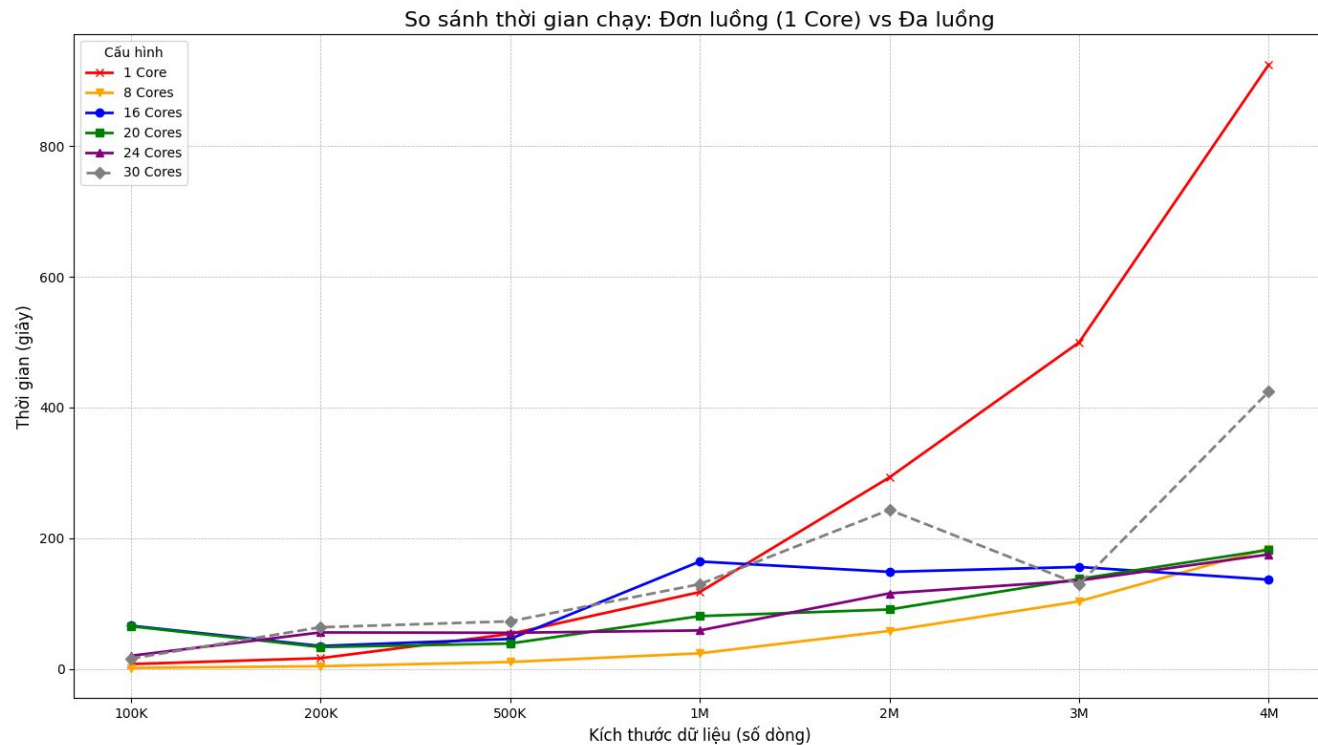


Kết quả



Với dữ liệu nhỏ, triển khai tuần tự là phù hợp hơn. Còn khi kích thước dữ liệu tăng lên hàng triệu bản ghi, việc sử dụng Spark trên HDFS giúp rút ngắn đáng kể thời gian xử lý mà vẫn bảo đảm chất lượng mô hình ở mức chấp nhận được.

So sánh giữa các mức cores



07

Kết luận

Dự án này đã chứng minh tính hiệu quả của việc kết hợp các công nghệ tiên tiến để giải quyết các thách thức về tốc độ và quy mô dữ liệu.

- **Lớp Lưu trữ (HDFS):** Đảm bảo tính **bền vững** và **khả năng chịu lỗi** cho các tập dữ liệu cực lớn, đồng thời tối ưu hóa hiệu suất thông qua nguyên tắc **Data Locality**.
- **Lớp Xử lý (Apache Spark):** Cung cấp nền tảng **xử lý tốc độ cao (In-Memory)**, cho phép chuyển đổi và phân tích dữ liệu hiệu quả, vượt trội so với các mô hình truyền thống.

Thành công trong việc minh họa và triển khai 2 loại thuật toán quan trọng trong môi trường phân tán:

Xử lý Dữ liệu Thô (Word Count): Đã mô tả và xác định được các giai đoạn chính của luồng dữ liệu phân tán (Map-Shuffle-Reduce), chứng minh khả năng xử lý khối lượng lớn văn bản một cách nhanh chóng.

Học máy Mở rộng (Random Forest): Đã phân tích cơ chế Song song của thuật toán này, cho phép xây dựng mô hình học máy phức tạp trên các cụm máy, giảm thời gian huấn luyện mà vẫn giữ nguyên độ chính xác.

Hạn chế

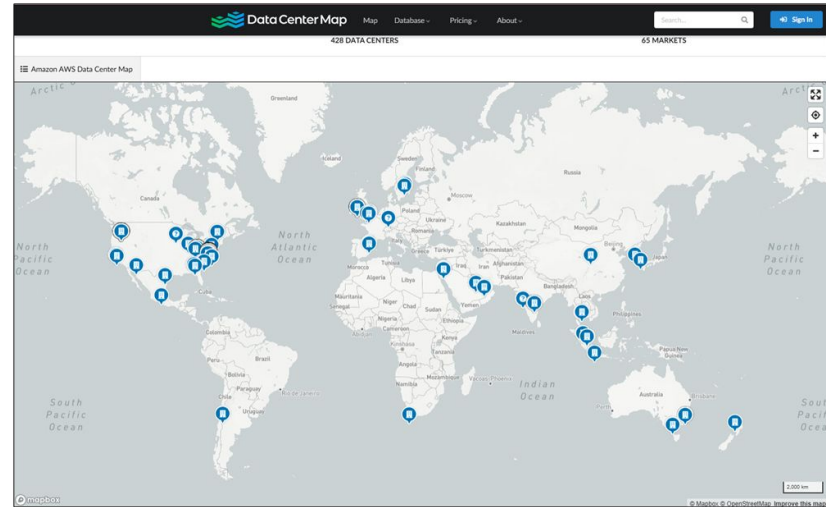
1. Hệ thống nhỏ (2 - 3 Nodes).
2. Dữ liệu chưa được gọi là Big Data (Rows, Columns).
3. Mỗi máy có CPU(xung nhịp, tiến trình,...) và RAM (bus, tốc độ đọc/ghi,...) khác nhau, tốc độ đọc dữ liệu trên ổ đĩa cũng khác nhau (tùy thuộc vào thế hệ). Nên việc chọn máy làm máy master cần nhiều thời gian hơn.

Hệ thống song song và phân tán hiệu quả

1. Phần mềm: Lựa chọn nền tảng (Hadoop, HDFS, Spark, Airflow, Kafka, hệ điều hành,...).
2. Phần cứng: Số lượng node, số lõi CPU, dung lượng RAM, ổ đĩa và băng thông mạng quyết định trực tiếp đến tốc độ xử lý và ngưỡng dữ liệu mà hệ thống có thể mở rộng.
3. Vị trí địa lý và kiến trúc mạng.



Các cụm máy lớn tại các công ty công nghệ



Data Center Map - Colocation, Cloud and Connectivity



**Cảm ơn thầy và
các bạn đã lắng
nghe**